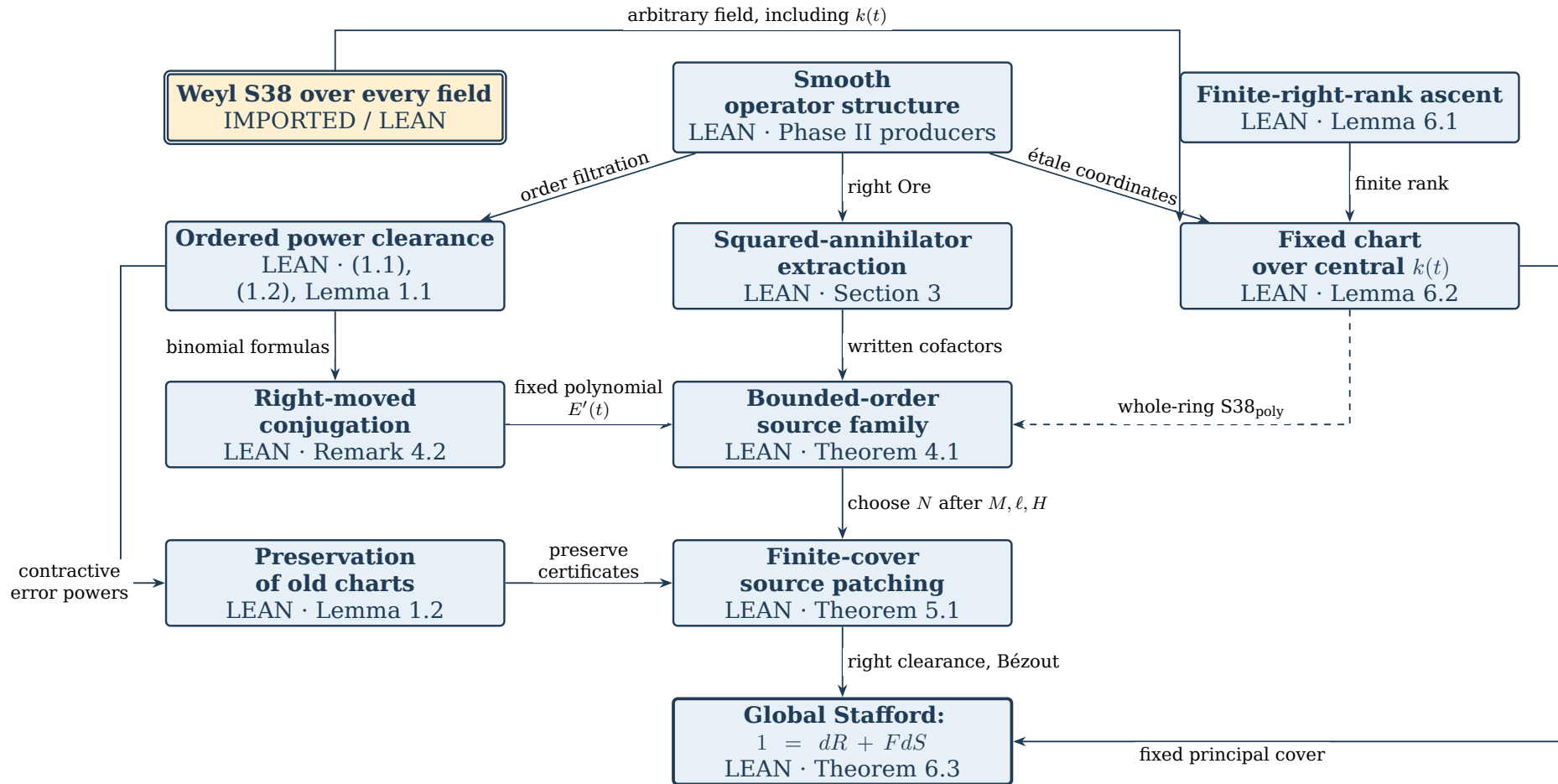


Global Stafford: the proof map

8 September 2026

Christopher Albert • Proof architecture and Challenge dossier

Every box links to its statement and Lean evidence below. Solid arrows show the main dependencies; the dashed arrow supplies a theorem hypothesis. Transitive infrastructure edges are omitted.



Blue solid boxes: project Lean proofs. Amber double box: imported Lean theorem. These tags refer to the recorded kernel verification, not human mathematical review. The map follows the research graph at b5654dd, using its Remark 4.2 formal variant.

1 The theorem and the meaning of its statement

This companion explains the finished formalization of the source-changing descent proof. It states the mathematical claim in TeX, compares every part with the Mathlib-only Challenge, and identifies the mechanisms that produce the global certificate. The canonical paper source is the Markdown note `notes/source-changing-descent-2026-09-07.md` in the research repository; this document is a TeX exposition and correspondence dossier, not a separate manuscript authority.

Theorem (Global Stafford). *Let k be any field of characteristic zero and let A be a smooth integral finitely generated commutative k -algebra. For every nonzero $d \in \mathcal{D}_k(A)$ there exist $F, R, S \in \mathcal{D}_k(A)$ such that*

$$1 = dR + FdS.$$

Here $\mathcal{D}_k(A)$ is the algebra of finite-order k -linear differential operators on A , and multiplication is composition in the written order.

Write $m_a(x) = ax$ for multiplication by $a \in A$. For $P \in \text{End}_k(A)$, put $[P, a] = P \circ m_a - m_a \circ P$. Define

$$\begin{aligned}\mathcal{D}_k^{\leq 0}(A) &= \{P : [P, a] = 0 \text{ for every } a \in A\}, \\ \mathcal{D}_k^{\leq r+1}(A) &= \{P : [P, a] \in \mathcal{D}_k^{\leq r}(A) \text{ for every } a \in A\}, \\ \mathcal{D}_k(A) &= \bigcup_{r \geq 0} \mathcal{D}_k^{\leq r}(A).\end{aligned}$$

An order-zero operator is multiplication by $P(1)$, since commuting with m_a gives $P(a) = aP(1)$. This is Grothendieck's finite-order definition. All operator ideals and modules in the argument are right-sided. Thus $d\mathcal{D} + Fd\mathcal{D} = \mathcal{D}$ means precisely the displayed certificate; R and S are right cofactors. The first generator remains the original d , and the second generator is the left multiple Fd of that same d .

There is no assumption that k is algebraically closed, no positive-dimensional restriction, and no global coordinate system on $\text{Spec } A$. For a polynomial algebra the statement specializes to the imported Weyl result. The proof does not prescribe a Bernstein-degree bound on F in the global case.

Exact source and verification scope. The paper revision is 9a096dc9f6c990bdfb3e8a24110989648c70b3f3, with SHA-256 22f047887464d268c1def305e697232826799bc112cec812e31e2cb8f8172d03. It is unchanged at research revision b5654dd. The independently replayed formal proof is b21883a5b3d8f46922713049c3b060523ea3a771. Release v1.0.3 uses unique submission module names. Phase I/full replay and both standard and canonical-Challenge comparisons passed with the dependency pins below. Phase I and II are complete in that record. Human expert review of the paper and its statement correspondence remains open.

2 TeX statement compared with GlobalStaffordChallenge.lean

The Challenge states the theorem on endomorphisms and explicitly requires finite order for each witness. The substantive proof instead uses the subalgebra of finite-order operators. These are two presentations of the same objects, connected by proved equivalences.

Mathematical statement	Challenge expression	Correspondence
Every characteristic-zero field k	$(k : \text{Type } u)$ $[\text{Field } k] [\text{CharZero } k]$	Arbitrary field in an arbitrary common universe; no closure assumption.
Smooth integral affine A/k	$[\text{CommRing } A] [\text{IsDomain } A]$ $[\text{Algebra } k A]$ $[\text{Algebra.Smooth } k A]$	Smooth means formally smooth and finitely presented in the pinned Mathlib. Over a field this supplies finite type.
$P \in \text{End}_k(A)$	$P : \text{Module.End } k A$	k -linear maps $A \rightarrow A$; multiplication is composition.
$[P, a] = Pm_a - m_aP$	commutator $P a$	Exactly the difference of those two ordered products.
$\text{ord } P \leq n$	$\text{IsOrderLE } n P$	The recursive commutator definition above, including its order-zero clause.
$d \in \mathcal{D}_k(A), d \neq 0$	$\text{IsDifferentialOperator } d \rightarrow d \neq 0 \rightarrow$	Both hypotheses are explicit; $d = 0$ is excluded.
$F, R, S \in \mathcal{D}_k(A)$	$\exists F R S : \text{Module.End } k A,$ followed by three $\text{IsDifferentialOperator}$ predicates	No witness is allowed to be an infinite-order endomorphism.
$1 = dR + FdS$	$(1 : \text{Module.End } k A)$ $= d * R + F * d * S$	1 is the identity map. Associativity changes parentheses only, not factor order.

The transport proof. `GlobalStafford.Assembly.isOrderLE_iff_mem_order` proves by induction on n that the Challenge predicate is membership in the library’s order filtration. `isDifferentialOperator_iff_mem_algebra` then quantifies over n . The theorem `challenge_of_twoGeneratorIdentity` places d in the intrinsic subalgebra, applies the substantive theorem, and coerces its witnesses back to endomorphisms. Coercion preserves 1, addition, and composition, and its injectivity preserves the nonzero hypothesis. No extra mathematical hypothesis is introduced.

The definitions are restated in `GlobalStafford/Assembly/ChallengeTransport.lean`; `GlobalStaffordSolution.lean` imports the substantive closure and uses the transported theorem. It never imports `GlobalStaffordChallenge.lean`. Comparator checks the Challenge and Solution statements in separate environments. Its equality check connects the two Lean statements; this TeX correspondence remains a mathematical reading task.

3 The two formal phases

Phase I proves every project-owned implication on concrete carriers. Its endpoint is `GlobalStafford.PhaseI.twoGeneratorIdentity_of_inputs`, taking a value of `GlobalStafford.Assembly.Inputs k A`. The fields specify a finite principal cover, operator localization and clearance, the whole-chart polynomial S38 property, chart and global domain structure, and right Ore data. These are explicit hypotheses, not undeclared axioms.

Phase II constructs every field from `Mathlib`, `AlgebraicAnalysis`, and the imported `Stafford38` theorem. The constructor `GlobalStafford.inputs` is fed to the Phase I endpoint by `GlobalStafford.universalStatement`. The unconditional statement therefore has only the field, characteristic, algebra, domain, and smoothness hypotheses of the theorem above.

Phase	Packages	Deliverable
I	WP-1-WP-10	Conditional descent and chart implications
II	WP-11-WP-18	Concrete producers and universal closure
III	WP-19	Challenge transport and independent replay

The record allows only `propext`, `Classical.choice`, and `Quot.sound` at both terminal endpoints. The single deliberate `sorry` is in the frozen Challenge template; it is absent from the Solution’s import closure. Optional AA-2-AA-4 library extractions concern proofs already present here and do not reopen Phase I or II.

4 Weyl S38 over the central parameter field

For every characteristic-zero field K , every $n \geq 0$, and every nonzero $a \in A_n(K)$, the imported theorem supplies $F, r, s \in A_n(K)$ with $1 = ar + Fas$. The field quantifier is essential: the chart producer invokes it at $K = k(t)$, not only at the original k .

The dependency is `itpplasma/stafford38-formal` at `e77e176c381ca2d6b20c030f9227f1b031415d2e`, declaration `Stafford38.universalStatement` in `Stafford38/FoundationClosure.lean`. It is a Lean theorem rebuilt against the root project’s shared `AlgebraicAnalysis` pin, not a literature axiom. Its own proof architecture remains upstream. The Global Stafford argument uses its universal certificate, not the upstream fixed-source strengthening. [Return to proof map](#)

5 Smooth operator structure

The Phase II producers establish the structural inputs on the actual algebra $\mathcal{D}_k(A)$. Finite-order operators extend to A_f , preserving order; the extension is injective for nonzero f in the domain A , and every localized operator has a left coefficient denominator. Commutator estimates then give right denominator clearance.

On an integral affine étale chart, coordinate derivations lift uniquely and commute, finite-order coordinate generation gives normal forms, and the leading-symbol calculation proves that the operator algebra has no zero divisors. Weyl right Ore and finite-rank transfer supply chart right Ore; injective localization and right clearance transfer it back to $\mathcal{D}_k(A)$. Smoothness supplies a finite principal cover carrying étale coordinate maps. These are the paper’s standard structural inputs, discharged in Lean rather than retained as literature assumptions.

Lean. `GlobalStafford/Assembly/Closure.lean`.

`GlobalStafford.exists_inputs`

[Return to proof map](#)

6 Finite-right-rank ascent

Let $T \subseteq S$ be domains, with T right Ore, and let S have finite positive right rank over T . If T satisfies S38, then S satisfies S38. For $0 \neq d \in S$, left multiplication by d induces an injective map of the finite-dimensional right $Q(T)$ -space $S \otimes_T Q(T)$. It is surjective, so clearing a right denominator gives $dy = a$ with $y \in S$, $0 \neq a \in T$. Then

$$1 = ar + Fas = d(yr) + Fd(ys).$$

No finite generation of S as a T -module is required.

Lean expresses finite rank as `FiniteRightSpan`: finitely many $s_i \in S$ span every $x \in S$ after multiplying x on the right by some nonzero denominator from T . This is precisely the form produced by chart normal forms and a finite generic fibre. The localized tensor object is used as a module; it is not assigned an unjustified ring structure.

Lean. `GlobalStafford/Chart/FiniteRankAscent.lean`.

`GlobalStafford.Chart.twoGeneratorIdentity_of_finiteRightSpan`

[Return to proof map](#)

7 The whole-ring hypothesis on one fixed chart

Let C be an integral affine algebra étale over $B = k[x_1, \dots, x_n]$. The coordinate map is injective by flatness and nontriviality. The lifted coordinate derivations define an injective Weyl action $T = A_n(k) \rightarrow S = \mathcal{D}_k(C)$. Normal forms identify S with $C \otimes_B T$ as a right T -module. Writing $L = \text{Frac}(B)$, the generic fibre $C \otimes_B L$ is a finite-dimensional field over L . Hence S has finite right rank over T , and Section 6 proves S38 for S .

The same construction works over $k(t)$ on the *same* chart. The formal scalar extension is

$$C_K = k(t) \otimes_k C, \quad D_K = \mathcal{D}_{k(t)}(C_K).$$

An injective map $\Theta : S[t] \rightarrow D_K$ sends P to its base change and the central polynomial variable to multiplication by t . Every element of D_K is in its image after multiplication by a nonzero scalar polynomial. Clearing the three witness denominators in a certificate yields

$$\text{S38}_{\text{poly}}(S) : \quad \forall, 0 \neq E \in S[t], \quad \exists, 0 \neq h \in k[t], \quad u, w, v \in S[t], \quad h = Eu + wEv.$$

This is a whole-ring statement, needed for the subsequently constructed squared input. A certificate for one previously chosen d would not suffice. The ring permits scalar t -denominators only; t is not differentiated. It is not $\mathcal{D}_k(\text{Frac}(C)(t))$. The étale morphism need not be finite.

Lean. `GlobalStafford/Chart/ChartDataOverK.lean`.

`GlobalStafford.Chart.s38Poly_chart`

[Return to proof map](#)

8 Ordered power clearance and contractive powers

Put $R = \mathcal{D}_k(A)$ and $C_f(P) = [P, f]$. If $\text{ord } P \leq r$, then $C_f^{r+1}(P) = 0$ and, for $n \geq r$,

$$Pf^n = \sum_{j=0}^r \binom{n}{j} f^{n-j} C_f^j(P) \in f^{n-r} R_{\leq r}.$$

The identity follows by induction from $Qf = fQ + [Q, f]$. Thus $f^{-a}P$ has a right denominator cleared by any f^m with $m \geq a + r$. For $E = f^L P$, $L > r$, induction gives

$$E^j \in f^{jL-(j-1)r} R_{\leq jr} \quad (j \geq 1).$$

In the induction step move the fixed-order P past the existing left coefficient power. Its order is r , not the increasing order of E^j . The subsets $f^m R$ are right ideals, not generally two-sided ideals.

Lean. `GlobalStafford/Operators/ContractivePowers.lean`.

`GlobalStafford.Operators.pow_eq_multiplication_pow_mul`

[Return to proof map](#)

9 Squared-annihilator extraction

Fix the current global source B and $0 \neq d \in R$. Right Ore supplies

$$c = da \neq 0, \quad (Bd)c = db_0.$$

For $e = chd$, suppose $1 = e^2 U + We^2 V$ in the chart ring. Set

$$F = B + Wch, \quad \alpha = ahd, \quad \beta = b_0hd.$$

The required certificate is the exact associative identity

$$1 = d(\alpha eU - \beta V) + Fd(eV).$$

Indeed $d\alpha = e$, $d\beta = Bde$, and $Fd = Bd + We$; expansion cancels $-BdeV + BdeV$ and leaves $e^2 U + We^2 V$. The square provides the additional e in the second right cofactor. Every occurrence of d remains in its specified position.

Lean. `GlobalStafford/Certificate/SquaredAnnihilator.lean`.

`GlobalStafford.Certificate.certificate_of_square`

[Return to proof map](#)

10 Right-moved polynomial conjugation

The formalization follows paper Remark 4.2. The right binomial identity is

$$f^n P = \sum_{j=0}^n \binom{n}{j} (-1)^j C_f^j(P) f^{n-j},$$

with terms $j > n$ omitted. In $R_f[t]$ define the finite polynomial

$$\rho_t(P) = \sum_j \binom{t}{j} (-1)^j C_f^j(P) f^{-j}.$$

It satisfies $\rho_N(P) = f^N P f^{-N}$ for $N \in \mathbb{N}$. For the already fixed Ore data c, d , put

$$E'(t) = c\rho_t(dc)\rho_{2t}(d), \quad e_N = cf^N d.$$

Then $E'(0) = (cd)^2 \neq 0$ and $e_N^2 = E'(N)f^{2N}$. A polynomial certificate for E' specializes away from the finitely many roots of its scalar denominator to a certificate for e_N^2 . The source witness is not conjugated in this variant, which simplifies its uniform order estimate. The full inverse- τ_t automorphism route in the paper is not an additional formal endpoint.

Lean. `GlobalStafford/Conjugation/RightMoved.lean`.

`GlobalStafford.Conjugation.evalNat_squaredInput`

[Return to proof map](#)

11 Bounded-order source families

Apply $S38_{\text{poly}}(R_f)$ to the fixed nonzero $E'(t)$, obtaining

$$h(t) = E'(t)u'(t) + w'(t)E'(t)v'(t), \quad 0 \neq h \in k[t].$$

At an admissible N , take

$$U = f^{-2N}u'(N)/h(N), \quad W = w'(N)/h(N), \quad V = f^{-2N}v'(N).$$

Since the scalar $h(N)$ is central, $1 = e_N^2 U + W e_N^2 V$. Section 9 gives the successful local source $B + W c f^N$. Clear the finitely many coefficients of $w'(t)c$ on the left:

$$w'(t)c = f^{-\ell}P(t), \quad P(t) \in R[t], \quad \text{ord } P_i \leq M.$$

Both ℓ and M are fixed before N . The binomial formula makes the source global for $N \geq \ell + M$:

$$F_N = B + f^{N-\ell-M}T_N, \quad T_N = h(N)^{-1} \sum_{j=0}^M \binom{N}{j} f^{M-j} C_f^j(P(N)), \quad \text{ord } T_N \leq M.$$

Each P_i has order at most M , so evaluating the central polynomial does not increase this bound. Increasing N increases coefficient precision while the correction order remains bounded. The local cofactors may have growing orders and pole sizes.

Lean. `GlobalStafford/Descent/BoundedOrderSource.lean`.

`GlobalStafford.Descent.exists_bounded_order_sources`

[Return to proof map](#)

12 Preserving every previously solved chart

Suppose an old chart R_g has $1 = dU_g + BdV_g$. Let the new successful global source be $F = B + f^L T$, where

$$L > M + \text{ord } d + \text{ord } V_g, \quad \text{ord } T \leq M.$$

Right-clear its new-chart certificate to $f^m = dA_0 + FdB_0$ in R . Put $P = TdV_g$, $E = f^L P$, and $r = M + \text{ord } d + \text{ord } V_g$. Then $dU_g + FdV_g = 1 + E$. Choose j with $jL - (j-1)r \geq m$; the contractive-powers lemma gives $(-E)^j = f^m W$ in R_g . With the finite sum $Z = \sum_{i < j} (-E)^i$,

$$(1 + E)Z = 1 - (-E)^j, \quad 1 = d(U_g Z + A_0 W) + Fd(V_g Z + B_0 W).$$

This is an identity in the chart ring itself. It uses neither completion nor an assumption that coefficient ideals are two-sided. At the next chart step, the actual finite orders of these new cofactors are read before choosing the next exponent.

Lean. `GlobalStafford/Certificate/OldChartProtection.lean`.

`GlobalStafford.Certificate.protect_old_chart`

[Return to proof map](#)

13 Finite-cover patching

Take a fixed finite principal cover $f_1, \dots, f_s$ supplied by smoothness, with $S38_{\text{poly}}$ on each chart. Start with $B = 0$ and no certificates. At step i , retain one global B that works on every earlier chart. The new chart supplies M, ℓ, H from Section 11. Only then choose N outside the zeros of H and so large that

$$N - \ell - M > M + \text{ord } d + \max_{j < i} \text{ord } V_j.$$

The new F_N solves chart i , and Section 12 supplies fresh certificates on every earlier chart for that same F_N .

After exactly s additions, one global F works everywhere on the cover. Right-clear the local cofactors to $f_i^{m_i} = dA_i + FdB_i$. These coefficient powers generate the unit ideal in A ; choose $a_i \in A$ with $\sum_i f_i^{m_i} a_i = 1$. Multiplication on the right and summation give

$$1 = d\left(\sum_i A_i a_i\right) + Fd\left(\sum_i B_i a_i\right).$$

This final Bézout argument uses the commutative coefficient algebra only at the stated point; it does not commute a coefficient through an operator.

Lean. `GlobalStafford/Descent/FiniteCoverPatching.lean`.

`GlobalStafford.Descent.twoGeneratorIdentity_of_charts`

[Return to proof map](#)

14 The terminal assembly

Smoothness supplies the finite étale principal cover, Section 7 supplies its whole-ring polynomial $S38$ hypotheses, and Section 13 gives the same-divisor certificate in the original $\mathcal{D}_k(A)$. These are exactly the ingredients assembled in `GlobalStafford.exists_in_puts`. The Phase II theorem then applies the Phase I implication to those constructed inputs.

The paper also derives torsion-module cyclicity and two-generation of right ideals. Those consequences are outside this repository's terminal formal scope. Historical G0–G14 routes, Bjork compression, and the failed uniform cofactor-lifting approaches are not dependencies of this proof.

Lean. `GlobalStafford/Assembly/Closure.lean`.

`GlobalStafford.universalStatement`

[Return to proof map](#)

15 Verification and reproducible dependencies

The repository report `docs/verification-results.json` records the isolated Phase I/full replay and Comparator acceptance on 9 September 2026. Both NanoDa and Lean accepted the solution. The report SHA-256 is `08b8dcd152806c5033316100a624d8c61a67f9e069c18ac649f3c6612779db67`. A second comparison using Palomar’s canonical Challenge compiler and protected alias also passed. Its outer `systemd` wrapper was unavailable locally; Landrun and the mathematical checks were retained. Hosted registry acceptance and human expert review are separate.

Lean `leanprover/lean4:v4.33.0`

Mathlib `db584cd6d46c92f209a44c0f1c829460d327499d`

AlgebraicAnalysis `4aae47967f6ba02ffe2f639ab06564c9a9d1ecc8`

Stafford38 formal `e77e176c381ca2d6b20c030f9227f1b031415d2e`

Lake uses the root manifest’s AlgebraicAnalysis revision for both projects. The verifier builds `Stafford38.FoundationClosure` and `Stafford38.LocalizedDifferentialCorollaries` against that shared pin. A newer upstream release is not substituted without a dependency work package.

```
lake exe cache get
lake build
scripts/verify.sh --phase-i
scripts/verify.sh
scripts/bootstrap-palomar-tools.sh
scripts/verify-palomar.sh
```

Run the verifier modes sequentially and retain their logs before the next mode overwrites `.lake/verification`. The paper’s 984 exact rational regression assertions check ordered identities on finite examples; they do not replace universal Lean proofs or human review.

16 Release companion and Zenodo description

This companion follows the Stafford38 Challenge dossier and proof-map supplement. Stafford38 v1.1.0 includes the stronger exact-source comparison and corrected proof architecture. Both its Comparator configurations passed NanoDa and Lean kernel checking on the controller host. Its concept DOI is [10.5281/zenodo.22390721](https://doi.org/10.5281/zenodo.22390721).

For Global Stafford, a source release can include this dossier’s TeX, clickable map, PDF and checksums, together with the verification report and pinned software source. The prepared Zenodo description is:

Lean 4 formalization of the identity $1 = dR + FdS$ for every nonzero finite-order k -linear differential operator d on a smooth integral affine algebra over any characteristic-zero field. The proof extends the formalized Weyl case by fixed étale charts, bounded-order source corrections, preservation of earlier chart certificates, and finite-cover descent. Phase II constructs all structural inputs from Mathlib and the pinned AlgebraicAnalysis and Stafford38 developments. The package includes the Challenge/Solution pair,

verification evidence, and a companion document with a clickable proof map, the statement correspondence, and the main proof mechanisms.

The formal repository is public under Apache-2.0. Release v1.0.3 fixes the submission module collision and records the completed replay. No hosted Palomar acceptance or human referee report is claimed.

A The actual Challenge file

This listing is read directly from the repository file at build time. Its deliberate final sorry is the statement placeholder for comparison.

```
1 import Mathlib.RingTheory.Smooth.Basic
2 import Mathlib.Algebra.Module.LinearMap.End
3
4 /-!
5 # Global Stafford for rings of differential operators
6
7 Let `k` be a field of characteristic zero and `A` a smooth integral
8 finitely generated `k`-algebra, the coordinate ring of a smooth integral
9 affine variety over `k`. Let `D_k(A)` be the ring of `k`-linear finite-order
10 differential operators on `A` in Grothendieck's sense: `k`-linear
11 endomorphisms `P` of `A` such that iterated commutators with multiplication
12 operators eventually vanish. Products in `D_k(A)` are compositions. The
13 statement is Stafford's same-divisor identity for this ring:
14
15 > for every nonzero `d ∈ D_k(A)` there exist `F, R, S ∈ D_k(A)` with
16 > `1 = d R + F d S`.
17
18 Both `d R` and `F d S` are ordered products in the noncommutative ring
19 `D_k(A)`, so `d D_k(A) + F d D_k(A) = D_k(A)` with the same right divisor
20 `d` in both summands. Over the polynomial ring `A = k[x_1, ..., x_n]` the ring
21 `D_k(A)` is the Weyl algebra and the statement is Stafford's Conjecture 3.8
22 (J. London Math. Soc. (2) 18 (1978), p. 438), proved in the separate
23 `stafford38-formal` development; this file states the extension to every
24 smooth integral affine variety.
25
26 The statement uses only Mathlib. Smoothness is Mathlib's `Algebra.Smooth`
27 (formally smooth and of finite presentation), which over a field agrees with
28 the usual notion. Differential operators are defined below without forming a
29 subalgebra, so that no closure proof enters the statement; the products are
30 those of `Module.End k A`, that is, composition of maps. The compared theorem
31 is `universalStatement`; its `sorry` is the deliberate hole filled by
32 `Solution.lean`.
33 -/
34
35 universe u
36
37 namespace GlobalStaffordChallenge
38
39 variable {k A : Type u} [CommRing k] [CommRing A] [Algebra k A]
40
41 /-- Multiplication by `a`, as a `k`-linear endomorphism of `A`. -/
42 def multiplication (a : A) : Module.End k A := LinearMap.mulLeft k a
43
44 /-- The commutator `[P, a] = P ∘ a - a ∘ P` of an endomorphism with a
45 multiplication operator. -/
46 def commutator (P : Module.End k A) (a : A) : Module.End k A :=
47   P * multiplication (k := k) a - multiplication (k := k) a * P
48
49 /-- `IsOrderLE n P` says that `P` is a differential operator of order at most
50 `n`: order `0` operators commute with every multiplication, and `P` has order
51 at most `n + 1` when every commutator `[P, a]` has order at most `n`. -/
52 def IsOrderLE : ℕ → Module.End k A → Prop
53   | 0, P => ∀ a : A, commutator P a = 0
54   | n + 1, P => ∀ a : A, IsOrderLE n (commutator P a)
55
56 /-- A finite-order `k`-linear differential operator on `A`. -/
57 def IsDifferentialOperator (P : Module.End k A) : Prop :=
58   ∃ n : ℕ, IsOrderLE n P
59
60 end GlobalStaffordChallenge
61
62 namespace GlobalStaffordChallenge
63
64 open GlobalStaffordChallenge
65
66 /-- Global Stafford: for every characteristic-zero field `k`, every smooth
67 integral finitely generated `k`-algebra `A`, and every nonzero differential
68 operator `d` on `A`, there are differential operators `F, R, S` on `A` with
69 `1 = d * R + F * d * S`, the products being compositions of endomorphisms. -/
70 def UniversalStatement : Prop :=
```

```

71  ∀ (k : Type u) [Field k] [CharZero k]
72    (A : Type u) [CommRing A] [IsDomain A] [Algebra k A] [Algebra.Smooth k A]
73    (d : Module.End k A),
74    IsDifferentialOperator d → d ≠ 0 →
75      ∃ F R S : Module.End k A,
76        IsDifferentialOperator F ∧ IsDifferentialOperator R ∧
77        IsDifferentialOperator S ∧
78        (1 : Module.End k A) = d * R + F * d * S
79
80  /-- The compared theorem. The proof is supplied by `Solution.lean`. -/
81  theorem universalStatement : UniversalStatement.{u} := by
82    sorry
83
84  end GlobalStaffordChallenge

```

B The actual Solution file and Comparator configuration

The Solution reaches the transported theorem through the substantive closure. The transport's recursive equivalences are explained in Section 2.

```
1 import GlobalStafford.Assembly.Closure
2
3 /-!
4 # Solution
5
6 Transport of the proved Global Stafford theorem to the Palomar statement.
7 The definitions of `Challenge.lean` are restated verbatim in
8 `GlobalStafford/Assembly/ChallengeTransport.lean` (this file does not import
9 `Challenge`); the theorem below has the exact name and type of the compared
10 declaration `GlobalStaffordChallenge.universalStatement`.
11 -/
12
13 namespace GlobalStaffordChallenge
14
15 universe u
16
17 /-- Global Stafford, in the Mathlib-only form of `Challenge.lean`. -/
18 theorem universalStatement : UniversalStatement.{u} :=
19   GlobalStafford.challengeStatement.{u}
20
21 end GlobalStaffordChallenge
22
23 #print axioms GlobalStaffordChallenge.universalStatement
```

```
{
  "challenge_module": "GlobalStaffordChallenge",
  "solution_module": "GlobalStaffordSolution",
  "theorem_names": [
    "GlobalStaffordChallenge.universalStatement"
  ],
  "permitted_axioms": [
    "propext",
    "Quot.sound",
    "Classical.choice"
  ],
  "enable_nanoda": true
}
```
